Performance Engineering
Project: Breadth-First Search in GAPBS

Group1
Sergiu Alexandru Macra s5524032
Cosmin Andrei Stamate s5600723

October 2026

Abstract

We looked at the breadth-first search (BFS) kernel of the GAP Benchmark Suite on two systems: our laptop (Intel Core i7-12650H, under WSL2) and one node of the Hábrók cluster (two AMD EPYC 7763). We measured it with perf, PAPI, gprof, the timers of the benchmark and a small counter tool we wrote ourselves, and we checked these tools against each other. Most of the time goes to the bottom-up step and to threads waiting at OpenMP barriers. By counting the work, we found that a search only looks at 4.7 % of the edges of the graph. Our model, based on this count and on the measured bandwidth, latency and memory-level parallelism, shows that one thread is limited by memory latency, the laptop with all threads by memory bandwidth, and the Hábrók node by synchronisation and the two sockets. We tried seven optimisations. Three of them gave a clear improvement: relabelling the vertices by degree (2.4 times faster on the laptop, 1.8 times on Hábrók for a larger graph), using fewer threads on one socket (2.1 times) and spreading the graph over both NUMA nodes (2.1 times when the threads are spread). Compiler flags, schedules, huge pages and prefetching stayed within the noise.

1 Introduction

Breadth-first search (BFS) is the typical example of the graph traversal dwarf. It is the main kernel of Graph500 [6] and one of the six kernels of the GAP Benchmark Suite (GAPBS) [1]. BFS does almost no arithmetic. Its speed depends on how fast it can get data from memory, and most of these accesses are random. Because of this, a model of BFS has to describe data movement, latency and synchronisation, and not floating-point operations.

The goal of the project was not to write the fastest BFS, but to go through the full performance engineering process with the GAPBS reference code:

1. analyse the program with more than one tool, and check that the tools give correct results,
2. explain the measurements with a model whose parameters we measured ourselves,
3. choose, implement and test optimisations based on the analysis and the model.

In our work plan we wanted to use a laptop and the HAICGU cluster. We did not get access to HAICGU, so we used the Hábrók cluster of the university instead. We also did more memory and OpenMP optimisations than planned, because the analysis showed that these parts take most of the time, and fewer compiler optimisations.

2 The application: BFS in GAPBS

GAPBS stores a graph in compressed sparse row (CSR) format: an index array with $n + 1$ pointers of 8 bytes, and a neighbour array with vertex IDs of 4 bytes. For an undirected graph

every edge is stored in both directions. Our main input is a Kronecker graph of scale 20 (`kron20`): $n = 2^{20}$ vertices, $m = 15{,}699{,}691$ undirected edges and 136 MB in memory. We generated it once and load it from a file in every run, because in the first naive runs generating and building the graph took 84 % of the time. To change the size of the working set we also used `kron22` (0.5 GB) and `kron24` (2.1 GB).

The BFS (`src/bfs.cc`) is a direction-optimising BFS [2]. It returns the parent of every vertex in the BFS tree. The important functions are:

- `TDStep` (top-down): every vertex in the frontier queue goes over its neighbours and takes the ones that are not visited yet, with an atomic compare-and-swap on `parent[v]`. The access to `parent[v]` is random. The loop over the queue uses a static schedule.
- `BUStep` (bottom-up): every vertex that is *not* visited yet goes over its neighbours until it finds one that is in the frontier bitmap, and then stops. This step goes over all n vertices. The loop uses `schedule(dynamic,1024)`.
- `QueueToBitmap` and `BitmapToQueue` convert the frontier when the direction changes.
- `InitParent` fills `parent` with the negative degrees at the start of every search.

`DOBFS` switches to bottom-up when the edges leaving the frontier are more than $1/\alpha$ of the edges that are not explored yet, and back to top-down when the frontier is smaller than n/β. The default values are $\alpha = 15$ and $\beta = 18$.

How much work is really done. We added a counter for the examined edges (`-DPE_COUNT_EDGES`). On average over 16 searches on `kron20`, one search does 4.25 top-down steps and 2.5 bottom-up steps. It examines 286,935 edges in top-down and 1,202,922 edges in bottom-up, which is only 4.7 % of the 31,399,382 directed edges. Bottom-up checks 1,659,885 vertices in total, and most of them find a parent before they have read one full cache line of neighbours. The usual metric for BFS is traversed edges per second (TEPS), which counts all m edges of the graph. We use it as well, but it overestimates the real work by about 20 times, so our model is based on the counted work (Section 6).

3 Systems and environment

Table 1: The two systems. Bandwidth and latency were measured by us (Section 6).

	Laptop	Hábrók node (`regular`)
CPU	Intel Core i7-12650H	2 × AMD EPYC 7763 (Zen 3)
Cores	6 P-cores + 4 E-cores (16 threads), WSL2 shows 10 vCPUs	2 × 64 cores, no SMT, 2 NUMA nodes
Caches	L1d 48 KiB per core, L3 24 MiB	L2 512 KiB per core, L3 16 × 32 MiB (one per CCD)
Memory	one DDR4-3200 channel (25.6 GB/s peak), 7.6 GiB visible in WSL2	503 GiB
STREAM triad, all / 1 thread	15.9 GB/s / 12.3 GB/s	327 GB/s / 34.7 GB/s
Load latency at 1 GB	376 ns	99 ns
OS	Ubuntu 24.04.1 on WSL2, Linux 6.18	AlmaLinux 9.8
Compiler	GCC 13.3, `-O3 -fopenmp`	GCC 11.5, `-O3 -fopenmp`
Transparent huge pages	`madvise` (default)	`always`
Tools	perf 6.18 (software events and sampling), gprof	PAPI 7.1 (built by us), pecount, gprof

Table 1 shows the two systems. All builds also use `-g -fno-omit-frame-pointer`, so that perf can record call graphs. This does not change the code of the loops.

Laptop under WSL2. WSL2 runs Linux in a Hyper-V virtual machine, and this limits what we can measure on the laptop:

- The hardware counters are not passed through to the virtual machine. All hardware events (cycles, instructions, cache and TLB misses) give *not supported*. Only software events and timer-based sampling work.
- The virtual machine shows 10 vCPUs and does not tell which ones are P-cores and which are E-cores, so we cannot choose between them.
- The memory addresses are translated twice (nested paging). This is probably one reason for the high memory latency of 376 ns, and it also adds noise.

The laptop has only one memory channel with a peak of 25.6 GB/s. STREAM reaches 15.9 GB/s with all threads, which is 62 % of the peak. In Assignment 6 we saw on the same laptop that STREAM is fastest with about four threads and gets slower with more threads. We wanted to boot a native Linux on the laptop to get hardware counters, but we did not manage this in time, so all hardware counters come from Hábrók.

Hábrók. The compute nodes have no `perf`, no `numactl` and no PAPI module, and `perf_event_paranoid` is set to 2, so we can only count events in user space. We therefore built PAPI 7.1 from source in our home directory. We also wrote `pecount` (`pe/bench/pecount.c`, about 200 lines), which counts the events of a program with `perf_event_open`, like `perf stat` does. All Hábrók runs used one full node (exclusive) through Slurm. Still, when several counters were open, they were only active 14 % to 26 % of the time. Probably other software on the node also uses the counters. For exact counts we therefore use PAPI regions with only a few events.

4 Measurement method and checking the tools

Changes to the code. All our changes are compile-time switches in `src/pe_opts.h`. Without them the code behaves like the original GAPBS. The only change in the default build is that every `#pragma omp parallel for` is split into `parallel` and `for`, which does the same thing. With the switches we can turn on PAPI regions inside every parallel region (the PAPI high-level API only counts the thread that calls it), the edge counter, `schedule(runtime)`, prefetching, huge pages, parallel first touch, relabelling and other values of α and β. A script builds 27 variants, and every variant has to pass the BFS verifier of GAPBS (`-v`) before we time it.

How we time. For every configuration we first do one run that we throw away, and then 5 runs of 24 searches each. The sources of the searches are chosen pseudo-randomly, but they are the same for all variants and both machines. We use the timer of GAPBS around one search. Inside one run the first searches are slow (Figure 1): on Hábrók the first five take between 99 and 156 ms, and after that about 4 ms. So we drop searches 1 to 5 of every run, which leaves 95 samples per configuration. As in the lectures, we report the median and the interquartile range (IQR) in ms per search, and GTEPS $= m/t_{\text{median}}$. The IQR is between 14 % (Hábrók, kron22) and 56 % (laptop, kron20) of the median. We only call a difference significant if it is bigger than the IQR of both configurations.

Checking the tools. Before we use a tool, we want to know that it measures the right thing. Table 2 shows our checks.

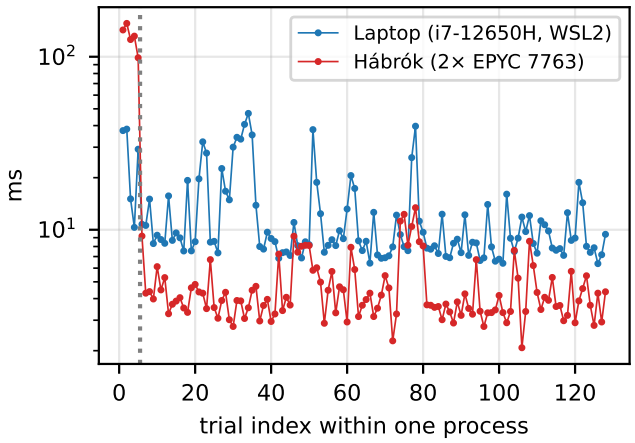


Figure 1: Time of every search inside one run (128 searches, log scale). The slow first searches on Hábrók and the outliers on the laptop are the reason why we drop searches 1 to 5 and use the median.

The verifier found a data race. Running the verifier on every variant was useful. With `schedule(runtime)` and `OMP_SCHEDULE=dynamic,100` or `guided`, *every* search gave a wrong BFS tree (0 of 8 correct on both machines), and `static,1000` was wrong in 4 of 8 searches on the laptop and in all 8 on Hábrók. The reason is that `BUStep` sets bits in the shared `next` bitmap with a normal, non-atomic read-modify-write of a 64-bit word. This is only safe when the block of every thread starts at a word boundary, so when the chunk size is a multiple of 64. The original `dynamic,1024` fulfils this without saying so. Plain `static` passed our tests, but only by luck, because its blocks also do not have to start at a word boundary. In the schedule experiments we therefore only used chunk sizes that are a multiple of 64.

5 Performance analysis

Baseline. On `kron20`, the original GAPBS with the default number of threads needs 8.58 ms per search on the laptop (median, IQR 4.77 ms, 1.83 GTEPS) and 2.85 ms on Hábrók (IQR 1.43 ms, 5.51 GTEPS). The Hábrók node has 12.8 times more cores and 20 times more memory bandwidth, but it is only 3 times faster. The main question of our analysis is why.

Hotspots. Table 3 shows where the laptop spends its time, measured with perf sampling (about 46,000 samples, 64 searches, all threads). Half of all CPU time is spent by threads that wait in the OpenMP runtime. One search has about ten short parallel regions (Section 2) and each of them ends with a barrier. The work inside them is not balanced: Kronecker graphs have a few vertices with a very high degree, and in bottom-up the cost of a vertex depends on how fast it finds a neighbour in the frontier. Of the useful work, `BUStep` is by far the biggest part. With `perf annotate` (`pe/results/wsl/perf_annotate_all.txt`) we looked at the instructions of `BUStep`. 39 % of its samples are on the instruction just after the load of the first neighbour ID. With timer sampling, the sample usually lands on the instruction that uses the result of a slow load, so this is the cache miss at the start of every neighbour list. Another 11 % are on the bit test of the bitmap and about 13 % on loads of the CSR index. So the time is spent on memory accesses, not on computation.

Hardware counters per function (Hábrók). With PAPI regions we can measure every function separately, which sampling on the laptop cannot do (Table 4). We used one thread, so that every region is fully counted. Even without any synchronisation, `BUStep` takes 78 % of the

Table 2: Checks of the measurement tools.

Check	Result	Conclusion
Timer resolution	`CLOCK_MONOTONIC`: 1 ns, one call takes 34 ns to 36 ns	small compared to a search (more than 1 ms)
Wall time from 3 tools (laptop, 64 searches)	GAPBS load + searches 0.679 s, perf `duration_time` 0.717 s, `/usr/bin/time` 0.69 s	they agree, the 38 ms difference is starting the program
CPU time from 2 tools	perf task-clock 6.00 s, `time` user + sys 6.05 s	within 1 %
pecount against perf	page faults 36,422 and 36,423	pecount gives the same result
gprof	100 % of the samples in `frame_dummy`	**not usable** for OpenMP: it does not assign the time of the OpenMP threads to the right functions
perf sampling	BUStep, InitParent and TDStep are found; libgomp has no symbols but is found by library	reliable
PAPI against pecount (Hábrók, 1 thread, 64 searches)	sum of the 5 PAPI regions divided by pecount: cache misses 99.1 %, references 97.2 %, cycles 95.0 %, instructions 90.3 %	they agree, the rest is code outside the regions (last loop, bitmap resets)
Known result: integer loop (PAPI)	5.000 instructions and 4.000 cycles per iteration	the loop has 5 instructions, `imul` (3 cycles) + `add` (1 cycle)
Known result: pointer chase over 1 GB (PAPI)	1.0007 cache misses per step	one miss per random load, as expected
Known results with pecount	loop: 4.99 instructions per iteration; chase: 2.4 misses per step	counters were shared (multiplexed), and the scaling is wrong when the program has phases, so we use PAPI regions
Timers on Hábrók	`/usr/bin/time` is not installed	cross-check only on the laptop

cycles. Both steps run at an IPC of about 1, while the core could do several instructions per cycle. In `TDStep`, 75 % of the examined edges give a cache miss, which is what we expect for the random accesses to `parent[v]`. In `BUStep` only a third of the checked vertices give a miss. The neighbour lists of vertices that follow each other are next to each other in the CSR array, so the hardware prefetcher can load many of them in time. The two conversion functions are simple bit operations with a high IPC, and they cost very little.

Branches also cost time. With one thread a search has 17.8 million branches, and 2.3 million of them are mispredicted (12.9 %, measured with pecount). Most of them come from the bit test and the early exit in `BUStep`, which depend on the data. A misprediction typically costs 10 to 20 cycles, so this can be a large part of the 87 million cycles of a search, although part of it overlaps with the waiting for memory.

The data TLB is not a problem on Hábrók: transparent huge pages are always on there, and one search only has about 4200 dTLB misses, compared to about 2 million random accesses. We also measured the counters with 128 threads, but these runs include the five slow first searches, which take most of the run time, so we do not use these numbers.

Scaling with the number of threads. Figure 2 shows the time per search for different numbers of threads. On the laptop the best time is with 8 threads (8.39 ms). With 10 threads it is not faster, and the IQR becomes much larger (12.3 ms). On Hábrók the time goes down

Table 3: Laptop, perf sampling (task-clock, 5 kHz), `kron20`, 64 searches including loading the graph.

Function	Samples	What it does
OpenMP runtime (`libgomp`)	50.0 %	threads waiting at barriers
`BUStep`	35.8 %	bottom-up step
`InitParent`	5.7 %	initialisation of every search
`TDStep`	3.1 %	top-down step
`BitmapToQueue` + `QueueToBitmap`	2.4 %	frontier conversion
`DOBFS` (last loop) and rest	3.0 %	

Table 4: PAPI per function, Hábrók, 1 thread, `kron20`, per search (64 searches). Misses are the generic event `PERF_COUNT_HW_CACHE_MISSES`.

Function	Cycles [M]	Share	IPC	Cache misses [k]	Misses per unit of work
`BUStep`	64.0	77.7 %	0.77	532	0.32 per checked vertex
`InitParent`	11.6	14.0 %	0.89	3.7	-
`TDStep`	3.6	4.4 %	1.01	216	0.75 per examined edge
`BitmapToQueue`	2.4	2.9 %	4.80	0.2	-
`QueueToBitmap`	0.8	0.9 %	1.59	2.5	-

until 48 to 64 threads that are bound to neighbouring cores (1.38 ms, 11.4 GTEPS), and then it goes *up* again, to 3.53 ms with 128 threads. With `OMP_PROC_BIND=close`, 64 threads fill exactly one socket. When the second socket is used, `parent` and the bitmaps have to be kept coherent between the two sockets, more threads have to meet at every barrier, and every thread has less work. Without binding, the best time is 2.45 ms with 48 threads.

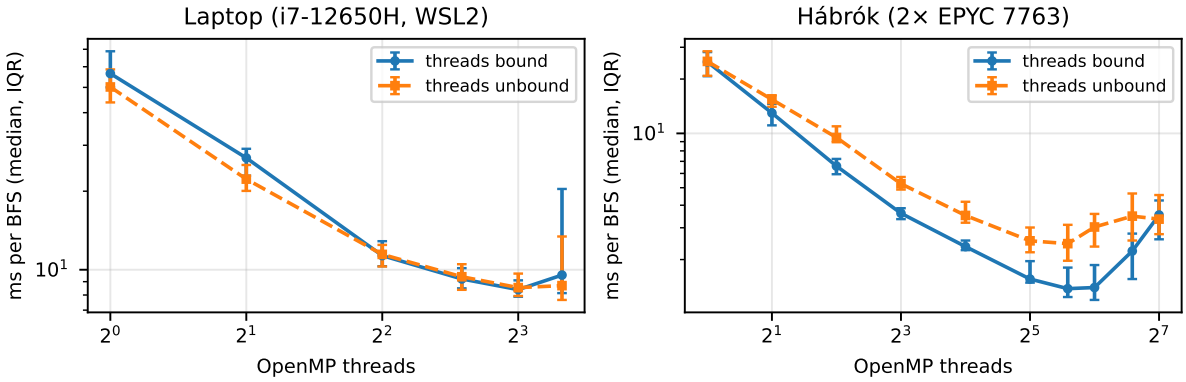


Figure 2: Time per search on `kron20` against the number of OpenMP threads, with the threads bound to neighbouring cores (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`) and not bound (median, error bars: IQR).

Memory hierarchy. Figure 3 shows the latency of dependent loads on both machines, measured with a pointer chase over a random cycle. The steps show the cache sizes. On Hábrók the latency stays at 13 to 15 ns up to 32 MiB (the L3 of one CCD) and is about 99 ns in main memory. On the laptop it goes up to 376 ns at 1 GB, probably because of TLB misses with nested paging. The working set of one `kron20` search is about 180 MB (Section 6). This is bigger than all caches of the laptop. On Hábrók it is bigger than the L3 of one CCD, but would fit in all 16 L3 slices together. A core can only use the L3 of its own CCD, so data cached by another CCD probably has to come over the interconnect, with a latency close to main memory.

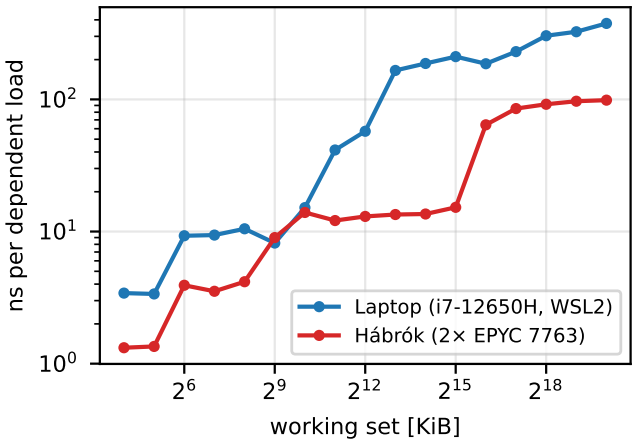


Figure 3: Latency of dependent loads against the size of the working set (one core).

6 Performance model

Which model. BFS does almost no floating-point operations, so a normal roofline in FLOP/byte [4] does not make sense here. If we use examined edges as the unit of work instead, the arithmetic intensity is only 1.49 million edges per 179 MB, so about 0.008 edges per byte. The ECM model needs a loop that always does the same work with predictable data transfers, so that in-core time and transfer time can be added. BFS has early exits that depend on the data, atomic operations, and the work changes in every level, so ECM does not fit either. We therefore used the information exchange model from the lectures: we count how many bytes have to be moved and how many random accesses have to be waited for, based on the work we counted and not on the size of the graph. Then we compute two lower bounds, one for bandwidth and one for latency, with parameters that we measured ourselves:

$$T_{\mathrm{bw}}(p) = \frac{Q}{\min(p\,B_1,\ B_{\mathrm{all}})}, \qquad T_{\mathrm{lat}}(p) = \frac{N_{\mathrm{rand}}\ L_{\mathrm{DRAM}}}{\mathrm{MLP}\cdot p}, \qquad T_{\mathrm{model}}(p) = \max(T_{\mathrm{bw}}, T_{\mathrm{lat}}).$$

Here Q is the amount of data of one search, B_1 and B_{all} are the STREAM triad bandwidth with one thread and with all threads, N_{rand} is the number of random cache line accesses, L_{DRAM} is the latency of a dependent load at 1 GB, and MLP is the number of misses that one core can have open at the same time (Little's law). T_{model} is a lower bound: it ignores synchronisation and assumes that all data comes from main memory.

Parameters. Q and N_{rand} come from the edge counts of Section 2 (Table 5). In bottom-up, every checked vertex costs at least one cache line of its neighbour list (usually the first neighbours are enough). In top-down, every examined edge costs 4 bytes for the neighbour ID and one random cache line of `parent`. The loops over `parent` (4 B per vertex), the CSR index (8 B per vertex) and the bitmaps are sequential. We measured the MLP with up to 32 pointer chases at the same time on one core. On the laptop the time per load goes down from 262 ns with one chain to 17 ns with 16 chains, so MLP = 15.7. This fits the 16 L1 miss status handling registers (MSHRs, also called fill buffers) of a Golden Cove core, which limit the number of open misses. On Hábrók the time per load goes down from 101 ns to 4.65 ns with 32 chains, so MLP = 21.7.

Model against measurement. Table 6 and Figure 4 compare the model with the measured times.

We see three different cases:

Table 5: Modelled amount of data of one `kron20` search.

Part	MB per search
`InitParent` (read index, write `parent`)	12.6
Bottom-up loops over all vertices, $2.5 \times (12n + n/8)$	31.8
Bottom-up: one neighbour line per checked vertex (1.66M × 64 B)	106.2
Top-down: 0.29M edges × (4 B + one random 64 B line)	19.5
Conversions and last loop	8.7
Total Q	**178.8**
Random line accesses N_{rand}	1.95 M

Table 6: Model bounds and measured median (threads bound).

Machine	p	T_{bw} [ms]	T_{lat} [ms]	T_{model} [ms]	measured [ms]	ratio
Laptop	1	14.6	46.7	46.7	56.5	1.21
Laptop	8	11.3	5.8	11.3	8.4	0.74
Hábrók	1	5.2	8.9	8.9	24.9	2.81
Hábrók	48	0.55	0.18	0.55	1.38	2.52
Hábrók	128	0.55	0.07	0.55	3.53	6.46

1. **One thread: limited by latency.** On both machines T_{lat} is larger than T_{bw}, so one core cannot have enough misses open to use the bandwidth. On the laptop the measured time is 21 % above the latency bound. On Hábrók it is 2.8 times the bound. This means the BFS only keeps about 7.7 misses open ($N_{\text{rand}} L_{\text{DRAM}}/T$), while the core could keep 21.7. Every checked vertex is a chain of loads that depend on each other (`parent[u]`, then the index, then the neighbour, then the bitmap word), so the out-of-order window only sees a few vertices at the same time. On the laptop the latency is much higher, so this limit is less visible there. To improve the model, we would have to use the MLP that BFS can really reach instead of the maximum of the core.

2. **Laptop with all threads: limited by bandwidth.** With 8 threads the latency bound is below the bandwidth bound of 11.3 ms. The measured 8.4 ms is even 26 % faster than this bound. Moving all 179 MB from main memory in 8.4 ms would need 21 GB/s, which is more than the 15.9 GB/s of STREAM, so part of the data comes from the 24 MiB L3. `parent` (4 MB), the bitmaps and part of the index fit there. The laptop is at its bandwidth limit, which explains why more threads do not help (Figure 2).

3. **Hábrók with many threads: neither bound.** Both bounds go down to about 0.5 ms, but the best measured time is 1.38 ms (2.5 times the bound), and with 128 threads it is 6.5 times the bound. The missing time is synchronisation (about ten barriers per search with up to 128 threads), unbalanced work, and coherence traffic between the two sockets. None of these are in a bandwidth and latency model. This explains why the Hábrók node is only 3 times faster than the laptop. It also shows that on Hábrók we should reduce the number of threads or the synchronisation, not the memory traffic.

Attainable performance. If we write the bound as GTEPS (m/T_{model}), the laptop can reach 1.4 GTEPS when all data comes from main memory. Because of the L3 we already reach 1.87 GTEPS, so there is not much more to gain without reducing Q. On Hábrók the bound is about 28 GTEPS for `kron20`, and our best result is 11.4 GTEPS. So Hábrók still has room, but only for optimisations that reduce synchronisation, place the threads and data better, or reduce Q with better locality.

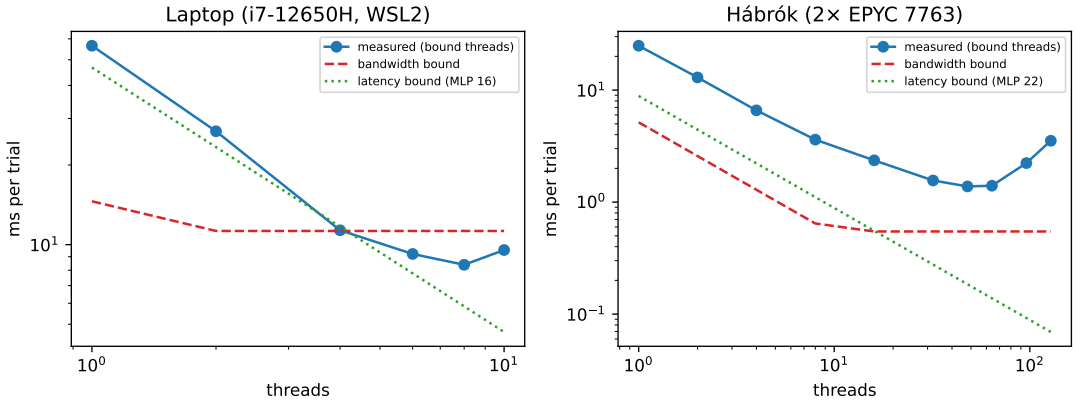


Figure 4: Measured time per search (threads bound) with the bandwidth and latency bounds of the model.

7 Optimisations

We chose every optimisation because of something we saw in the analysis or the model, and every variant passes the BFS verifier. Figure 5 shows the speedups on `kron20` and Table 7 the medians and IQRs. As explained in Section 4, a difference only counts as significant if it is bigger than the IQR of both configurations.

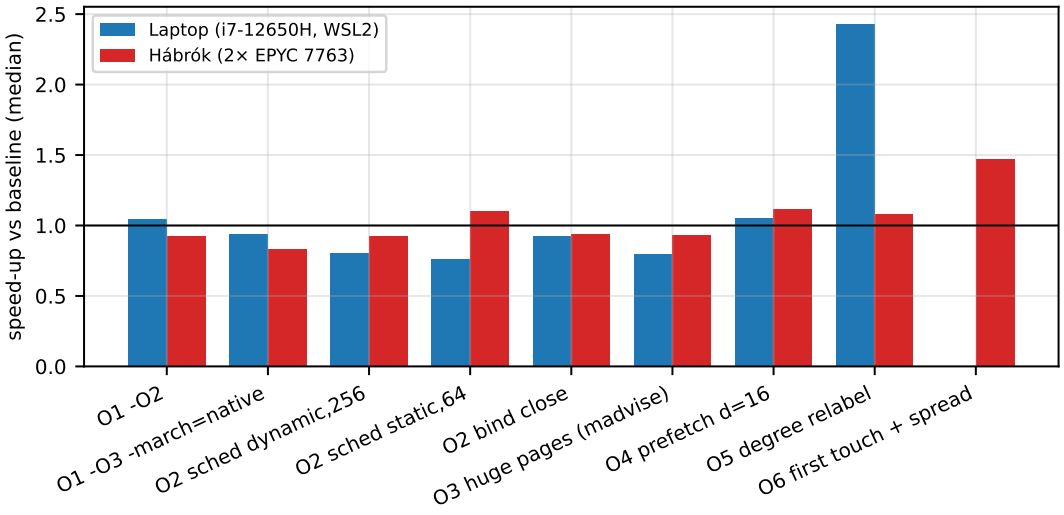


Figure 5: Speedup of some variants compared to the original code (`kron20`, all threads, ratio of the medians).

7.1 O1: Compiler options and vectorisation

Why. This is the easiest thing to try and it was in our work plan. *Result.* `-O0` is about 2.5 times slower than `-O3` on both machines, `-O1` already gets most of the speed back, and `-O2` and `-O3` are the same within the IQR. `-march=native` is not faster (9.16 ms against 8.01 ms on the laptop and 3.43 ms against 2.75 ms on Hábrók, both within the IQR). The GCC report (`-fopt-info-vec-all`) explains why: none of the important loops is vectorised. The inner loop of `BUStep` is not vectorised because of *control flow in loop* (the `break`), its outer loop because of *multiple nested loops*, and `TDStep` because the atomic compare-and-swap *clobbers memory*. To vectorise BFS one would need a different formulation of the algorithm. For the same reason

Table 7: Median [IQR] in ms per search. Original code: `-O3`, default schedule and number of threads. Bold: significant improvement.

Variant	Graph	Laptop	Hábrók
Original	kron20	8.58 [4.77]	2.85 [1.43]
O1 `-O0` / `-O2` / `-O3 -march=native`	kron20	20.3 / 8.22 / 9.16	7.03 / 3.07 / 3.43
O2 `static,64` / `dynamic,256` (both loops)	kron20	11.3 / 10.7	2.59 [3.21] / 3.08
O2 8 (laptop) or 48 (Hábrók) threads, bound	kron20	8.39 [1.23]	**1.38 [0.57]**
O3 THP `never` / `madvise` / patch / `always`	kron20	9.98 / 11.75 / 10.72 / 8.24	- / - / 3.05 / 2.97
O3 huge page patch vs. original	kron22	50.2 [43.2] vs. 71.1 [58.5]	12.83 vs. 12.14
O4 prefetch distance 16	kron20	8.15 [3.70]	2.55 [1.57]
O5 relabelling by degree	kron20	**3.53 [1.77]**	2.64 [3.13]
O5 relabelling vs. original	kron22	25.1 [12.8] vs. 71.1 [58.5]	**6.67 [1.35]** vs. 12.14 [1.66]
O5 relabelling vs. original	kron24	-	33.7 [17.3] vs. 38.2 [5.7]
O6 original vs. first touch, spread	kron20	-	4.00 [1.17] vs. **1.94 [1.18]**
O6 original vs. first touch, spread	kron22	-	12.33 [1.21] vs. **10.96 [1.35]**
O7 $\alpha = 15, \beta = 9$ / $\alpha = 5, \beta = 9$	kron20	8.17 / 37.8	2.42 [1.34] / 2.76

we did not try loop transformations like unrolling or unroll-and-jam: the loops of BFS have trip counts that depend on the data and early exits, and `-O3` already unrolls where it can. The model also shows that the bottleneck is the memory, so the number of instructions does not matter much once `-O1` has removed the obvious overhead.

7.2 O2: OpenMP schedule, binding and number of threads

Why. Half of the samples are in the OpenMP runtime (Table 3), and on Hábrók the model does not explain a factor of 2.5 to 6.5 (Section 6). *What we changed.* `schedule(runtime)` for both loops, tested with chunk sizes that are a multiple of 64 (Section 4), thread binding, and the number of threads. *Result.* No schedule is better than the original combination (static for top-down, `dynamic,1024` for bottom-up). Using the same schedule for both loops is slower on the laptop, and the best one on Hábrók (`static,64`, 2.59 ms) is within the IQR. Binding alone also makes no significant difference. The number of threads does: on Hábrók, 48 threads bound to neighbouring cores (1.38 ms) are **2.1 times faster** than the default of 128 unbound threads. They all run on one socket, so there is no coherence traffic between the sockets and the barriers are smaller. On the laptop the best is 8 of the 10 vCPUs, but the difference is not significant.

7.3 O3: Huge pages

Why. The latency curve (Figure 3) goes up when the TLB is not big enough anymore, and `BUStep` and `TDStep` access a CSR of 136 MB at random. *What we changed.* `madvise(MADV_HUGEPAGE)` on the CSR arrays and the `pvector`s before they are first used. With `/proc/self/smaps_rollup` we checked that 127 MB of the program then uses 2 MB pages, and 0 without the patch. *Result.* On `kron20` all four laptop configurations are within each other's IQR. On `kron22` the patch makes the median on the laptop 29 % lower, but the IQRs are above 40 ms, so this is not significant. Hábrók already uses transparent huge pages (`always`), so the original code already gets huge pages and the patch changes nothing: about 4200 against 4900 dTLB misses per search, which is very little compared to about 2 million random accesses. On the laptop, where TLB misses make the memory latency higher (Figure 3), the effect we hoped for is hidden by the noise of WSL2.

7.4 O4: Software prefetching in top-down

Why. The random accesses to `parent[v]` in `TDStep` are limited by latency, and prefetching a few neighbours ahead should give more open misses at the same time. *Result.* No prefetch

distance is significant on either machine. The best is distance 16 on Hábrók, 11 % faster but within the IQR. The analysis already predicted this: top-down is only 3 % of the samples and 11 % of the modelled data, so even perfect prefetching could only save a few percent (Amdahl's law). In the bottom-up step, where most time is spent, the loads for different vertices are already independent, so the out-of-order core can overlap them.

7.5 O5: Relabelling the vertices by degree

Why. The model shows that most of the data is one neighbour line per checked vertex in bottom-up, plus the random accesses to `parent`. If we number the vertices by decreasing degree (`-DRELABEL`, using `RelabelByDegree` from GAPBS), the vertices with a high degree get small IDs that are next to each other, which gives better locality. Because the neighbour lists are sorted, these vertices, which are most likely to be in the frontier, are also at the start of every neighbour list. *Result.* This is the best optimisation: **2.4 times faster** on the laptop (`kron20`) and **1.8 times** on Hábrók for `kron22`. On the laptop the median for `kron22` is even 2.8 times lower, but there the IQR of the original code is too large to call it significant. Our edge counter shows why relabelling works: a search examines 35 % fewer edges on `kron20` and 48 % fewer on `kron22`, because bottom-up finds a parent in the frontier earlier. So it reduces both Q and the work. On `kron24` the gain (1.13 times) is not significant. *Note.* The relabelling itself takes 0.46 s for `kron20`. This is not included in the search time and is paid back after about 90 searches on the laptop. After relabelling the vertices have other IDs, so the pseudo-random sources are not the same as in the original runs.

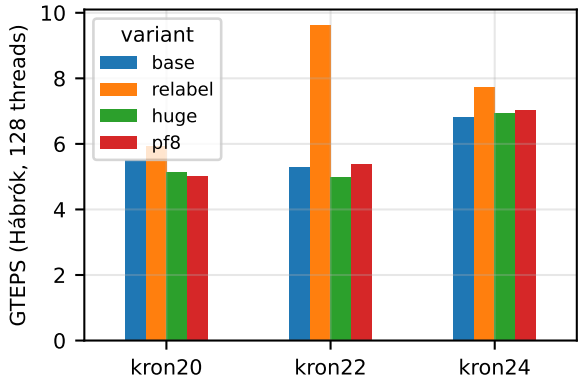


Figure 6: GTEPS on Hábrók for larger graphs. Relabelling helps most on `kron22`, huge pages and prefetching make no difference on this node.

7.6 O6: First touch for NUMA

Why. GAPBS loads the graph with one `read()`, so one thread touches every page first, and on Hábrók the whole edge array (125 MB) ends up on one socket. The threads on the other socket then read everything from remote memory, and only the memory controllers of one socket are used. `numactl` is not available on Hábrók, so we changed the code: `-DPAR_FIRST_TOUCH` first writes zeros to the array in parallel and then reads the file into it. *Result.* The placement matters as soon as the threads are on both sockets. With `OMP_PROC_BIND=spread` the original code becomes slower, 4.00 ms [IQR 1.17] on `kron20`, because half of the threads read everything remotely. With the parallel first touch it takes 1.94 ms [IQR 1.18], which is **2.1 times faster**, and also 1.5 times faster than the default configuration. On `kron22` first touch is 11 % faster with spread binding (from 12.33 to 10.96 ms, significant). Without binding the gains are smaller and within the IQR, because the operating system places the threads differently every time. On

`kron24` the difference is not significant, and we did not find out yet why the effect is gone for the largest graph.

7.7 O7: The switching parameters α and β

Why. α and β decide how much of the work is done in each of the two very different steps. *Result.* We tried 3 values for each (3×3), and the default values are already close to the best. With $\alpha = 5$ the search stays in top-down for too long and is 4.4 times slower on the laptop. The best settings ($\alpha = 15$, $\beta = 36$ on the laptop and $\alpha = 15$, $\beta = 9$ on Hábrók) are within the IQR of the default.

8 Discussion and conclusion

What limits BFS. The profiles, the edge counts and the model all point in the same direction. One core is limited by latency, because it cannot keep enough misses open. The laptop with all threads is limited by bandwidth, and it is even faster than its main memory bound because of the L3. The 128-core Hábrók node is limited by neither of them, but by synchronisation (about ten barriers per search) and by the two sockets. The optimisations confirm this. The ones that helped reduce the data or the work (relabelling), the number of threads that have to synchronise (fewer threads on one socket), or the remote memory traffic (first touch). The ones that only change the instructions (compiler options, vectorisation) or a small part of the program (prefetching in top-down) did not help. In most cases the analysis and the model told us in advance which optimisations could work.

Limitations.

- The laptop runs under WSL2: no hardware counters, P-cores and E-cores cannot be told apart, and a lot of noise (IQR up to 56 %). This is why we use the median and only accept differences larger than the IQR.
- The model assumes that all data comes from main memory, which is why the laptop is faster than its bound. A version that takes the caches into account would use the L3 bandwidth for the 4 MB `parent` array.
- The counters on Hábrók were multiplexed, and the counter runs with 128 threads include the slow first searches, so we only use the single-thread counters as exact numbers.
- Relabelling changes the source vertices of the searches.
- `kron20` is a small graph for 128 cores. On `kron22` and `kron24` the GTEPS are higher, because there is more work per barrier (6.8 GTEPS on `kron24`).

Future work. Hardware counters on the laptop with a native Linux, which would also allow a top-down microarchitecture analysis; the comparison with Apple Silicon from our work plan; a bit test without branches in `BUStep` against the 12.9 % mispredictions; splitting the bottom-up loop by NUMA node; and merging the short parallel regions of one BFS level to have fewer barriers, similar to loop fusion. The last one is about the biggest cost we found.

9 Workload distribution

Name	Parts
Cosmin Andrei Stamate	laptop measurements, Sections 3, 4, 5, 7.1, 7.2
Sergiu Alexandru Macra	Hábrók measurements, Sections 1, 2, 6, 7.3 to 7.7
Both	scripts, analysis, Section 8

Table 8: Workload

References

[1] S. Beamer, K. Asanović, D. Patterson. *The GAP Benchmark Suite*. arXiv:1508.03619, 2015.

[2] S. Beamer, K. Asanović, D. Patterson. *Direction-Optimizing Breadth-First Search*. SC'12, 2012.

[3] J. D. McCalpin. *STREAM: Sustainable Memory Bandwidth in High Performance Computers*. `https://www.cs.virginia.edu/stream/`.

[4] S. Williams, A. Waterman, D. Patterson. *Roofline: an insightful visual performance model for multicore architectures*. CACM 52(4), 2009.

[5] PAPI: Performance Application Programming Interface, ICL, University of Tennessee. `https://icl.utk.edu/papi/`.

[6] Graph500 Benchmark Specification. `https://graph500.org`.

Code and data

All code, scripts and raw results are in `https://github.com/CosminStamateAndrei/gapbs` (folder `pe/`). The file `pe/README.md` explains how to reproduce every result.